

Analysis: A Digging Problem

The setup of this problem, where you can go left, right and down but never up hints that some dynamic programming solution is the way to go, but the details are a bit involved.

When we're on a row we need to know which are the rock cells that were dug previously so that we know how much we can move left or right. This means that we could have a state in our algorithm be $(i, j, \text{air_holes})$ where i is the current row, j is the current column and air_holes is the set of cells on row i that were dug previously or were empty. Filling the values for these states in the whole matrix would take exponential time as air_holes can take on as many as 2^C values. This would be enough to solve the small input, but for the large we need to improve our algorithm a bit.

First let's observe that it only makes sense to dig out cells that will form a connected empty zone. After we fall one row, we'll be able to use just the current zone of empty boxes. Now our state is $(i, j, \text{start}, \text{end})$ where start is the starting column index of the current empty zone and end is the index of the zone's end column. This idea yields a polynomial solution, as we have $O(R * C^3)$ possible states and there are at most C^2 different states that we can create on the next row.

But we can improve on this solution. It doesn't make sense to change directions after we started to dig; if we're moving right, it doesn't matter how many empty cells we have on the left. This changes the state to $(i, j, \text{dir}, \text{count})$ where dir is the current direction (left or right), and count is the number of empty cells in that direction. This reduces the state space to $O(R * C^2)$.

The implementation details are somehow tricky, and you have to make sure you don't fall more than F steps -- a detail we've skipped here. There are many possible ways to implement this problem, some of which result in much simpler code than others. We encourage you to download and study various correct implementations from the scoreboard.

Before the contest began, we evaluated this problem as the second easiest in the contest; but the many details needed to solve it resulted in this problem having the second-smallest number of successful solutions.

Something related

If you are among our older contestants, this problem may bring back sweet memories of the classic game [Lode Runner](#), and perhaps memories of many happy days associated with it.

Analysis: Crazy Rows

Reformulation of the problem

It is easy to see, for each row, only the position of the last '1' matters. We can re-formulate the problem

CR: Given a list of numbers ($a_0, \dots, a_{N-1}$). You are allowed to swap adjacent numbers. Find the minimum number of swaps needed so that the i -th number is at most i in the final configuration.

The well known special case

Perhaps many of you know the following special case.

CR*: Given a permutation ($x_0, \dots, x_{N-1}$) of the numbers 0 to $N-1$. You are allowed to swap adjacent numbers. Find the minimum number of swaps needed in order to sort the list in increasing order.

Perhaps you also know the complete solution to CR*. It is very simple and elegant, and important to our problem. So we produce here.

Solution (to CR*). Define the *disorder* of the list to be the number of pairs $i < j$, where $x_i > x_j$. In one swap (of adjacent numbers), the total number of disorder is changed by exactly one. (Check it!) Therefore, let the disorder of initial configuration be D , you need at least D swaps. D swaps is also sufficient -- as long as the list is not sorted, there exist adjacent pairs in wrong order. You can swap any of such pairs and decrease the disorder by 1. $\diamond$

In particular,

(1) One type of optimal solutions of CR* involves first to swap the number 0 all the way to the left, then let it stay there forever.

Solution to our problem

Imagine we know which of the a_i 's will finally go to position 0, and which one will go to position 1, etc., then we can simply use the algorithm for CR*. But there might be multiple candidates for a single position. For example, there might be several i 's such that $a_i = 0$, and even some $a_i = -1$.

Below is the judge's C++ solution. $b[i]$ is the "decoded" position where $a[i]$ will be in the final configuration. The algorithm says: For those candidates for position 0, pick the leftmost one. Then in the rest, for those candidates for position 1, pick the leftmost one. And so on.

```
// -1 means no position is assigned for a[j].
for(i=0;i<N;i++) b[j]=-1;
for(i=0;i<N;i++) {
    for(j=0;j<N;j++) if(b[j]<0 && a[j]<=i) {
        b[j]=i; break;
    }
}
```

```

    }
}
int r=0;
for(i=0;i<N;i++) for(j=i+1;j<N;j++)
    if(b[i]>b[j]) r++;
// output r as the answer

```

Note that, once the $b[i]$'s are fixed, you only need to count the disorders as in CR*; no real swapping needs to be simulated.

Proof of our solution

The key observation is, for multiple candidates for position 0, you will never need to swap any two of them. Suppose you do swap, say u and v , both are at most 0. I can simply *ignore* it, and *pretend* that they are swapped (i.e., exchange the roles of u and v thereafter). The final configuration is still a good one. Thus we proved that, for all the candidates for position 0, the leftmost one, call it u^* , will finally go to position 0.

Now, imagine we have decoded the final positions for every one. Then (1) tells us that there is one solution where we first move u^* all the way to the left, and never worry about it again. Therefore we now face the next question: Which of the rest of the numbers should go to position 1?

This is exactly the same problem, but with one number fewer. $\diamond$

Analysis: Stock Charts

It will come as no great surprise, but the author of this problem came up with it while reading a local newspaper's end-of-year economics summary, seeing a number of overlaid charts in it and then wondering how to minimize the number of charts needed.

Charts and DAGs

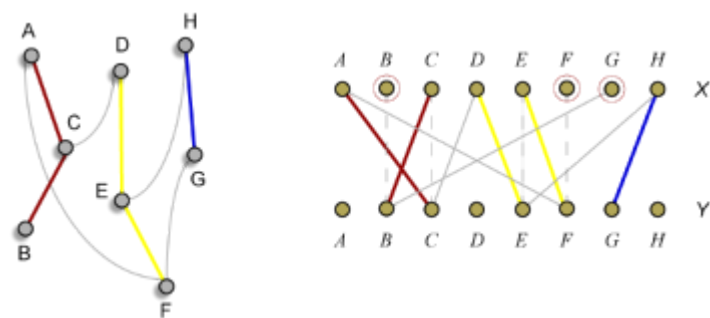
Consider two simple charts: A and B. They can be related to each other in one of three ways: all of A's values can be strictly less than all of B's values ($A < B$), in which case they can appear on the same overlaid chart; their lines can cross ($A \setminus B$), in which case they can't appear on the same overlaid chart; or all of A's values can be strictly greater than all of B's values ($A > B$), in which case they can appear on the same overlaid chart.

Given this sort of relationship we can construct a graph, where the nodes are simple charts and there is an edge from A to B iff $A > B$. This gives us a directed, acyclic graph that is its own (non-reflexive) transitive closure. Any directed path in the DAG represents a valid composite chart. To solve the problem, then, we want to find the minimum number of paths that we need so that all nodes are part of exactly one path.

How would we find the paths? We may start from a chart that is relatively high, then find one below it, and keep adding lower charts, until we cannot find more. This completes our first overlaid charts. We start the same process for the second path, and so on. In any step, there might be several choices for the next chart we can use. In order to minimize the number of paths, we need to make a good choice in each step.

Now, behold, the Aha moment of this problem.

Solution from maximum matching



For the DAG with n points, we make a bipartite graph with n points on each side. Draw an edge from X_A to Y_B if the relation $A > B$ holds, i.e., B can be the next chart below A. Observe (yes, you really need to see it, instead of hear it!) how any path in the DAG corresponds to a series of edges in the bipartite graph; and how any matching of the bipartite graph corresponds to a way to partition the DAG into paths. Any unmatched point X_A on the left side corresponds to the lowest point on a path (the lowest chart on an overlaid graph). Each path has exactly one such point. We want to minimize the number of paths, the same as minimizing the number of unmatched points on the left side. That is, we want to find the maximum matching in a bipartite graph.

Here is the judge's solution in C++.

```
namespace Solver {
int N,K;
bool cbn[111][111]; // can be next
int prev[111];
bool visited[111];

bool FindNextDfs(int a) {
    if(a<0) return true;
    if(visited[a]) return false;
    visited[a]=true;
    for (int i=0;i<N;i++) if(cbn[a][i]) {
        if(FindNextDfs(prev[i])) {
            prev[i]=a;
            return true;
        }
    }
    return false;
}

int Solve(const vector<vector<int> >& stock) {
    N=stock.size(); K=stock[0].size();
    int i,j,k;
    for(i=0;i<N;i++) {
        for(j=0;j<N;j++) {
            if(i==j) cbn[i][j]=false;
            else {
                cbn[i][j]=true;
                for(k=0;k<K;k++)
                    if(stock[i][k]<=stock[j][k]) cbn[i][j]=false;
            }
        }
    }
    memset(prev, -1, sizeof(prev));
    int ret=0;
    for(i=0;i<N;i++) {
        memset(visited, 0, sizeof(visited));
        if(!FindNextDfs(i)) ret++;
    }
    return ret;
}
```

Note that this is indeed the bipartite matching program. We named the variables as if we are really constructing the set of paths and unaware of the bipartite graph. In fact, it's a worthy exercise to go over this without bipartite matching in your mind.

This completes the solution of our problem. But we may continue with more stories.

Theoretical background

In combinatorics, DAGs are called *partially ordered sets*, or *posets*. A directed path is called a *chain* in the poset. An independent set in the DAG, corresponding to a set of points where no '>' relation holds between any two of them, is called an *anti-chain*. Our problem is then, given a poset, find the minimum number of chains needed to cover all the points.

If we see an anti-chain of size α , we need at least α chains to cover the set, because each chain can contain at most one of these points. Suppose we find the maximum anti-chain to be of size

α^* , we know the answer must be at least α^* . Is this enough though?
We are ready to introduce one of the classical theorems in combinatorics.

Theorem (Dilworth 1950) In a poset, the minimum number of chains needed to cover the whole set equals the size of the biggest anti-chain.

Dilworth's theorem is closely related to other classical theorems in combinatorics. In fact it is equivalent to Hall's marriage theorem on bipartite graphs, and the max-flow-min-cut theorem.

The number in Dilworth's theorem is (naturally) called the width of the poset. Our algorithm above thus finds the width of a poset. Interested readers might find, in our input file, perturbed copies of the following posets:

- The complete Boolean lattice: All the 2^k subsets of a k -element set, where $A > B$ if B is a subset of A . All the $\lceil k/2 \rceil$ -subsets form a maximum anti-chain of size $\binom{k}{\lceil k/2 \rceil}$, and indeed we can partition the Boolean lattice into this many chains. (This is called the Sperner's theorem.)
- All the integers from 1 to n , $A > B$ if A is a multiple of B . The set of all the prime numbers seems to be a big anti-chain. But it is not big enough. We leave it as an exercise to prove that the width of this poset is $\lceil n/2 \rceil$.

More information

[Partially ordered set](#) - [Dilworth's theorem](#)

Analysis: Watering Plants

This problem involves finding circles that enclose other circles. The problem of finding a circle surrounding a set of points is the fairly well-known minimal enclosing circle problem, but changing the points to circles makes the problem slightly trickier.

One solution is to do a binary search to find the minimum sprinkler radius. This reduces the problem to the problem of determining whether two sprinklers of a given radius can cover all the plants. To solve this, we can make the assumption that any sprinkler used in the solution either:

- covers exactly one plant, or
- the boundary of the sprinkler touches the boundary of at least two of the plants it covers.

This assumption is safe because if a sprinkler covers more than one plant but does not have two plants on its boundary, the sprinkler can be shifted and rotated, while still covering the same plants, until it does. Given this assumption, we can create a set of candidate sprinkler locations including:

- a sprinkler centered on each plant, and
- for each pair of plants, the set of sprinklers covering those plants and touching their boundary (there are 0, 1, or 2 of these per pair.)

Then we check every pair of candidate sprinklers, and see if any of them together cover every plant.

A second solution is to directly find the minimum sprinkler radius. To do this, we can use a slightly different simplifying assumption -- that every sprinkler either:

- covers exactly one plant (using the same radius as the plant),
- covers two plants which touch the edge of the sprinkler and whose centers are collinear with the center of the sprinkler, or
- covers three plants which touch the edge of the sprinkler.

This assumption is safe because any other sprinkler can be shrunk to a sprinkler of smaller radius that covers the same set of plants. We try each set of plants of size 1, 2 or 3, create the corresponding sprinkler from the three cases above, and check its radius and which set of plants it covers. Then we find the pair of sprinklers that covers every plant using the minimum maximum radius.

Finding the circle which touches 3 given circles is harder than the equivalent problem for 3 points. Here are three possible approaches:

1. The set of points where a sprinkler can be centered in order to touch two plants is a hyperbola, so we could algebraically compute the intersection of two of those hyperbolae.
2. We can use a gradient-descent approach to find numerically the point minimizing the function from potential centers of sprinklers to the radius required for a sprinkler centered at that location to cover all three plants.
3. We can subtract from the radius of each of the three plants the radius of the smallest plant, then compute an inversion about that plant's center. Then we find appropriate tangents to the two inverted plants, re-invert to find the corresponding circle, and add back the radius of the smallest plant.